

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения и технологий

Учебная дисциплина «Метрология, стандартизация и сертификация (в
информационных технологиях)»

Отчет

по лабораторной работе №1
«Метрики размера программ»

Подготовил:

Студенты гр. 410902
Кавцевич В. А.
Габрукович Р. С.

Проверил:

Болтак С. В.

Минск 2026

Цель: Изучить метрики размера программ на примере метрики Холстеда.

Задание: Написать программу-парсер. Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Для анализа взять код на заданном языке. На основе анализа программа должна рассчитать метрики Холстеда согласно примера 1 в материале выше: 6 базовых и 3 расширенных. Результат работы программы - таблица, как в примере, в которой должны отображаться 6 основных метрик Холстеда. Отдельно выводятся 3 расширенные метрики.

Код анализируемой программы на JavaScript:

```
class Stats {
  constructor(numbers) { this.numbers = numbers; }
  get count() { return this.numbers.length; }
  sum() {
    let total = 0;
    for (const n of this.numbers) total += n;
    return total;
  }
  average() {
    if (this.count === 0) throw new Error("Пустой массив");
    return this.sum() / this.count;
  }
}

function classify(value) {
  if (value < 0) return "отрицательное";
  else if (value === 0) return "ноль";
  else return "положительное";
}

function describeGrade(score) {
  switch (true) {
    case score >= 90: return "отлично";
    case score >= 75: return "хорошо";
    case score >= 60: return "удовлетворительно";
    default: return "неудовлетворительно";
  }
}

function collatzSteps(start) {
  let n = start, steps = 0;
  while (n !== 1) {
    if (n % 2 === 0) n = n / 2;
    else n = 3 * n + 1;
    steps++;
    if (steps > 1000) break;
  }
  do { steps = steps + 0; } while (false);
}
```

```

    return steps;
}
const double = (x) => x * 2;
const buildReport = (numbers, extra) => {
    const stats = new Stats(numbers);
    return { total: stats.sum(), average: stats.average(), count: stats.count, ...extra };
};
function inspectObject(obj) {
    const keys = [];
    for (const key in obj)
        if (typeof obj[key] === "number") keys.push(key);
    const copy = { ...obj };
    if ("temp" in copy) delete copy.temp;
    return keys;
}
function safeRun(fn, arg) {
    let result = null;
    try { result = fn(arg); }
    catch (err) { console.log("Ошибка:", err.message); }
    finally { console.log("Проверка завершена"); }
    return result;
}
function filterPositive(numbers) {
    const out = [];
    for (let i = 0; i < numbers.length; i++) {
        if (numbers[i] < 0) continue;
        out.push(numbers[i]);
    }
    return out;
}
function main() {
    const data = [4, -7, 0, 12, 5, -3, 9];
    console.log("=== Классификация ===");
    for (const n of data) console.log(n, "—", classify(n));
    console.log("=== Оценки ===");
    const scores = [95, 80, 62, 40];
    for (const s of scores) console.log(s, "—", describeGrade(s));
    console.log("=== Числа Коллатца ===", collatzSteps(27));
    console.log("=== Отчёт ===");
    console.log(buildReport(filterPositive(data), { tag: "v1" }));
    console.log("=== Ключи объекта ===");
    console.log(inspectObject({ a: 1, b: "x", c: 2, temp: 99 }));
    console.log("=== Безопасный запуск ===");
    safeRun(() => new Stats([]).average());
    console.log("Удвоенное 21:", double(21));
}

```

```

const flag = data.length > 0 ? "непусто" : "пусто";
console.log("Массив:", flag);
}
main();

```

Результаты расчета базовых метрик Холстеда представлены в таблице 1.

Таблица 1 – Результаты расчета базовых метрик Холстеда.

j	Оператор	flj	i	Операнд	$f2i$
1	Stats	3	1	numbers	11
2	{}	24	2	this	6
3	constructor	1	3	length	3
4	.	31	4	total	4
5	"="	19	5	0	11
6	;	59	6	n	12
7	count	5	7	Пустой массив	1
8	return	15	8	value	3
9	sum	3	9	отрицательное	1
10	for	5	10	"ноль"	1
11	"+="	1	11	положительное	1
12	average	4	12	score	4
13	if-else	8	13	true	1
14	"===="	4	14	90	1
15	throw	1	15	отлично	1
16	new	3	16	75	1
17	Error	1	17	хорошо	1
18	"/"	2	18	60	1
19	classify	2	19	удовлетворительно	1
20	<	3	20	неудовлетворительно	1
21	describeGrade	2	21	start	2
22	switch-case- default	5	22	steps	6
23	">="	3	23	1	3
24	collatzSteps	2	24	2	4
25	while	1	25	3	2
26	"!=="	1	26	1000	1
27	%	1	27	false	1
28	*	2	28	x	2
29	"+"	2	29	extra	2
30	"++"	2	30	stats	4
31	>	2	31	obj	4
32	break	1	32	keys	3
33	do-while	2	33	key	3
34	double	2	34	"number"	1

35	()	3	35	copy	3
36	"=>"	3	36	"temp"	1
37	buildReport	2	37	temp	2
38	"..."	2	38	arg	2
39	inspectObject	2	39	result	3
40	[]	8	40	null	1
41	typeof	1	41	err	2
42	push	2	42	console	14
43	in	1	43	Ошибка:	1
44	delete	1	44	message	1
45	safeRun	2	45	Проверка завершена	1
46	fn	2	46	out	3
47	try-catch- finally	3	47	i	5
48	log	14	48	data	4
49	filterPositive	2	49	4	1
50	continue	1	50	7	1
51	main	2	51	12	1
52	"_"	2	52	5	1
53	?:	1	53	9	1
			54	==== Классификация ====	1
			55	—	2
			56	==== Оценки ====	1
			57	scores	2
			58	95	1
			59	80	1
			60	62	1
			61	40	1
			62	s	3
			63	==== Числа Коллатца ====	1
			64	27	1
			65	==== Отчёт ====	1
			66	tag	1
			67	"v1"	1
			68	==== Ключи объекта ====	1
			69	a	1
			70	b	1
			71	"x"	1
			72	c	1
			73	99	1
			74	==== Безопасный запуск ====	1
			75	Удвоенное 21:	1

			76	21	1
			77	flag	2
			78	непусто	1
			79	пусто	1
			80	Массив:	1
$\eta_1 =$ 53		$N_1 =$ 276	$\eta_2 =$ 80		$N_2 =$ 184

Словарь программы $\eta = \eta_1 + \eta_2 = 53 + 80 = 133$.

Длина программы $N = N_1 + N_2 = 276 + 184 = 460$.

Объем программы $V = N * \log_2(\eta) = 460 * \log_2(133) = 3245,43$

Программа-парсер написана на языке Python с использованием библиотеки Tkinter для графического интерфейса. Результатом работы программы является вывод таблицы операторов и операндов, а также результата расчёта базовых и расширенных метрик Холстеда. Результат работы программы представлен на рисунках 1–4.

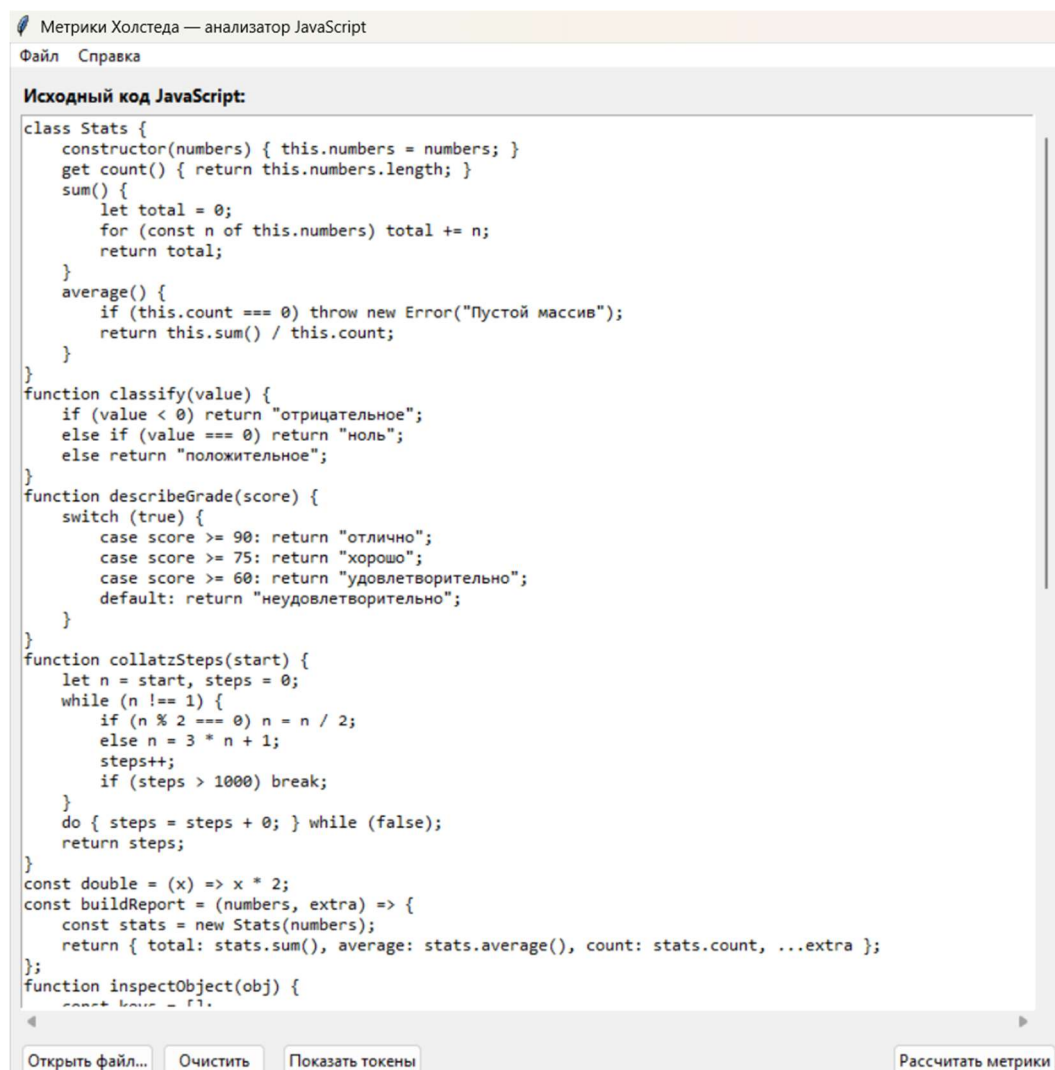


Рисунок 1 – Поле для ввода кода

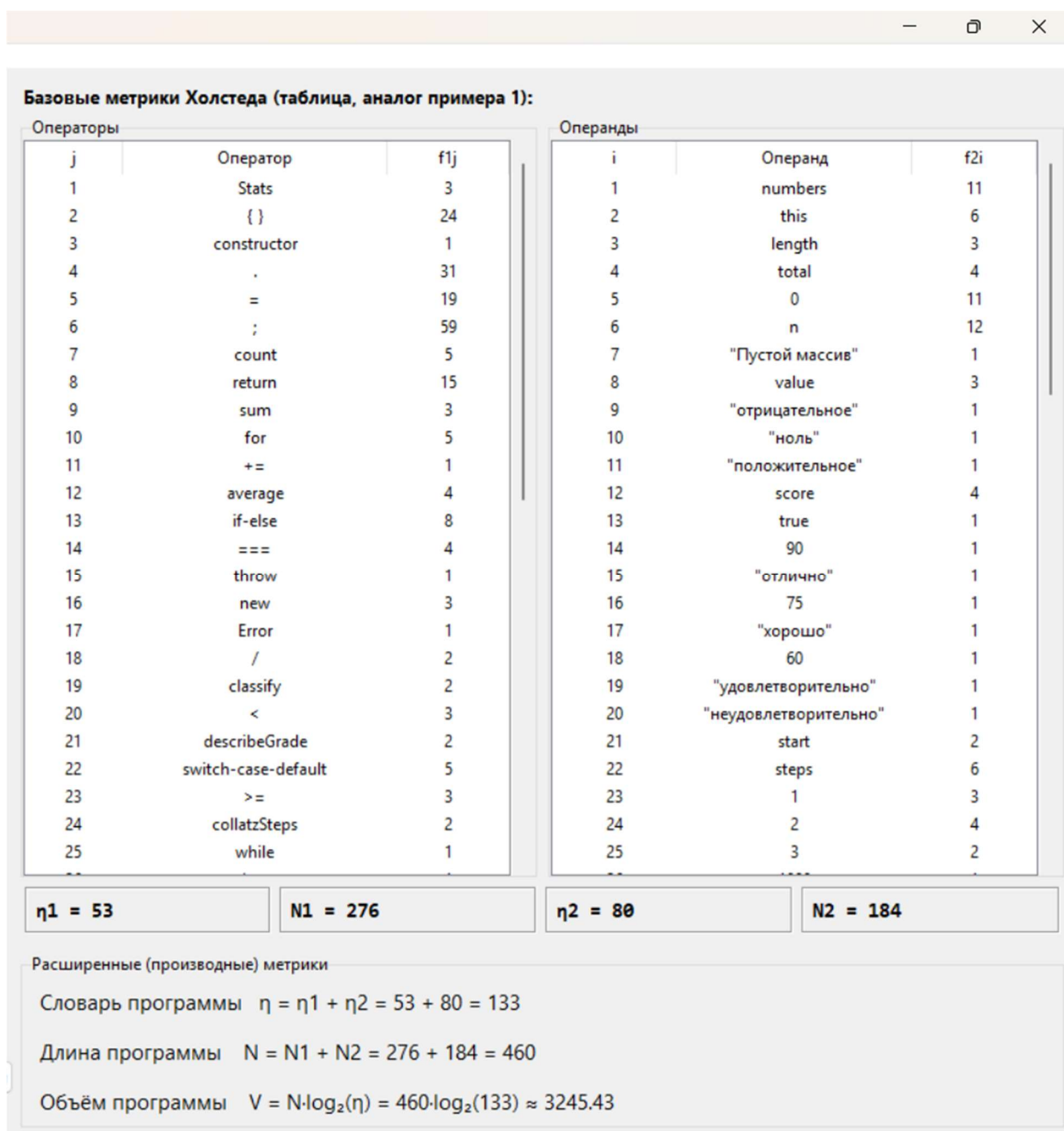


Рисунок 2 – Результат анализа кода

Операторы			Операторы		
j	Oneparop	f1j	j	Oneparop	f1j
1	Stats	3	25	while	1
2	{ }	24	26	!=	1
3	constructor	1	27	%	1
4	.	31	28	*	2
5	=	19	29	+	2
6	;	59	30	++	2
7	count	5	31	>	2
8	return	15	32	break	1
9	sum	3	33	do-while	2
10	for	5	34	double	2
11	+=	1	35	()	3
12	average	4	36	=>	3
13	if-else	8	37	buildReport	2
14	===	4	38	...	2
15	throw	1	39	inspectObject	2
16	new	3	40	[]	8
17	Error	1	41	typeof	1
18	/	2	42	push	2
19	classify	2	43	in	1
20	<	3	44	delete	1
21	describeGrade	2	45	safeRun	2
22	switch-case-default	5	46	fn	2
23	>=	3	47	try-catch-finally	3
24	collatzSteps	2	48	log	14
25	while	1	49	filterPositive	2
50	continue	1			
51	main	2			
52	-	2			
53	? :	1			

Рисунок 3 – Результат подсчета операторов

Операнды			Операнды		
i	Операнд	f2i	i	Операнд	f2i
1	numbers	11	25	3	2
2	this	6	26	1000	1
3	length	3	27	false	1
4	total	4	28	x	2
5	0	11	29	extra	2
6	n	12	30	stats	4
7	"Пустой массив"	1	31	obj	4
8	value	3	32	keys	3
9	"отрицательное"	1	33	key	3
10	"ноль"	1	34	"number"	1
11	"положительное"	1	35	copy	3
12	score	4	36	"temp"	1
13	true	1	37	temp	2
14	90	1	38	arg	2
15	"отлично"	1	39	result	3
16	75	1	40	null	1
17	"хорошо"	1	41	err	2
18	60	1	42	console	14
19	"удовлетворительно"	1	43	"Ошибка:"	1
20	"неудовлетворительно"	1	44	message	1
21	start	2	45	"Проверка завершена"	1
22	steps	6	46	out	3
23	1	3	47	i	5
24	2	4	48	data	4
25	3	2	49	4	1
26	...	...	50	...	...
Операнды			Операнды		
i	Операнд	f2i	i	Операнд	f2i
49	4	1	74	"=== Безопасный запуск ==="	1
50	7	1	75	"Удвоенное 21:"	1
51	12	1	76	21	1
52	5	1	77	flag	2
53	9	1	78	"непусто"	1
54	"=== Классификация ==="	1	79	"пусто"	1
55	"_"	2	80	"Массив:"	1
56	"=== Оценки ==="	1			
57	scores	2			
58	95	1			
59	80	1			
60	62	1			
61	40	1			
62	s	3			
63	"=== Числа Коллатца ==="	1			
64	27	1			
65	"=== Отчёт ==="	1			
66	tag	1			
67	"v1"	1			
68	"=== Ключи объекта ==="	1			
69	a	1			
70	b	1			
71	"x"	1			
72	c	1			
73	99	1			
74	...	...			

Рисунок 4 – Результат подсчета операндов